

Retro Tron Battle

Server-Dokumentation

Ein Netzwerk-Lightcycle-Duell für echte Retro-Computer (Atari, Commodore 64, Schneider/Amstrad CPC) mit serverseitiger Darstellung.

Gebaut für [Classic Computing 2026](#) in Celle (10.–11. Oktober 2026, Halle 10 / CD-Kaserne).

Dieses Dokument beschreibt den Python-Server (tron_server.py), die beiden Teilnahme-Methoden für Retro-Clients, die verwendeten Hardware-Interfaces, wie man ohne echte Hardware testet, und wie man das Setup um weitere Plattformen erweitert.

1. Überblick

Retro Tron Battle ist ein Mehrspieler-Lightcycle-Spiel (im Stil von "Tron"), bei dem echte Retro-Computer über ihre Netzwerk-Erweiterungen gegeneinander antreten. Die Besonderheit: die Retro-Rechner zeichnen das Spielfeld nicht selbst - sie senden nur die Steuerungsrichtung. Das komplette Spielfeld wird zentral auf dem PC gezeichnet, der den Server betreibt (über ein pygame-Fenster, gut geeignet für Beamer/große Monitore bei einer Veranstaltung).

Aktuell unterstützte Plattformen:

- **Atari XL/XE** mit [FujiNet](#)-Netzwerkkarte - verbindet sich per echtem TCP-Socket (Port 6502)
- **Commodore 64** mit [Meatloaf](#)-Interface (Standard-Client) - verbindet sich per HTTP-Polling (Port 8080)
- **Commodore 64** mit [WiC64](#)-Interface (experimenteller Client, User-Port statt IEC-Bus) - ebenfalls per HTTP-Polling (Port 8080)
- **Schneider/Amstrad CPC** mit [M4-Board](#) - verbindet sich ebenfalls per HTTP-Polling (Port 8080)

Warum unterschiedliche Methoden pro Plattform? Das ist keine Design-Entscheidung, sondern eine technische Notwendigkeit - siehe Abschnitt 3.

1.1 Architektur in Kürze

- Der Server (**tron_server.py**) führt die komplette Spiellogik aus: Bewegung, Kollisionserkennung, Punktestand.
- Ein **pygame-Fenster** zeigt das Spielfeld, Besucherfotos, Firmenlogos und einen Demo-Scroller an.
- Jeder Retro-Client tut fast nichts außer: Joystick lesen, Richtungswechsel senden, gelegentlich nachfragen, ob das Spiel vorbei ist.
- Ergebnisse werden fortlaufend in eine CSV-Datei geschrieben und beim Serverstart wieder eingelesen (Punktestand bleibt über Neustarts erhalten).
- Alle Pfade (Font, Logos, Ergebnis-CSV, Foto-Cache) werden relativ zum Speicherort von `tron_server.py` aufgelöst, nicht zum aktuellen Arbeitsverzeichnis - das Repo funktioniert also unverändert, egal von wo aus der Server gestartet wird.
- Konvention: Jeder Client liest den **ersten Joystick-Port** seines Rechners (Atari `STICK(0)`, CPC `JOY(0)`, C64 `PEEK(56321) / $DC01`).

2. Spielablauf

- Ein Spieler verbindet sich (TCP oder HTTP, siehe Abschnitt 3) und sendet optional eine Besucher-PIN für sein Event-Foto.
- Der Server wartet, bis ein zweiter Spieler sich anmeldet.
- Sobald zwei Spieler da sind: kurzer Countdown im Fenster (3, 2, 1, GO!) - Richtungswechsel werden schon entgegengenommen, aber noch niemand bewegt sich.
- Nach GO! startet die eigentliche Bewegung, mit einstellbarer Geschwindigkeit (TICK_RATE).
- Das Spiel endet bei einer Kollision (Wand, eigene Spur, gegnerische Spur) - Sieg, oder Unentschieden bei gleichzeitigem Crash.
- Ergebnis wird an beide Clients gesendet, in die CSV geschrieben, und der Server kehrt in den Wartezustand zurück (Demo-Scroller läuft wieder los).

3. Zwei Teilnahme-Methoden

Nicht jede Retro-Netzwerklösung kann eine dauerhafte TCP-Verbindung aus BASIC heraus offen halten. FujiNet (Atari) kann das zuverlässig - Meatloaf und WiC64 (beide C64) sowie das M4-Board (Amstrad CPC) sind dagegen auf einfache, einzelne HTTP-Anfragen ausgelegt. Ein roher Socket ist dort entweder technisch gar nicht aus BASIC möglich (M4, ohne Z80-Assembler) oder hat sich im Praxistest über viele Testrunden als unzuverlässig erwiesen (Meatloaf). Beide Wege laufen serverseitig durch exakt dieselbe Spiellogik (`run_game()`) - es ist nur eine andere "Vordertür".

3.1 Methode 1: Rohes TCP (Port 6502)

Verwendet z.B. vom Atari mit FujiNet. Eine Verbindung bleibt für die gesamte Spieldauer offen.

Client → Server

Befehl	Beschreibung
HELLO <PLATTFORM> <NAME> [PIN]	Anmeldung, PIN optional (Event-Foto-ID)
MOVE <U D L R>	Richtungswechsel (Kehrtwende in die eigene Spur wird ignoriert)
BYE	Verbindung/Spiel beenden

Server → Client

Nachricht	Beschreibung
WAIT	Verbunden, wartet auf Gegner
START <w> <h> <x1> <y1> <x2> <y2> <spielernr> <plat1> <plat2>	Steuerung darf jetzt beginnen
END WIN <PLATTFORM> <NAME>	Spiel vorbei, jemand hat gewonnen
END DRAW	Unentschieden (gleichzeitiger Crash)
STATS GAMES <n> ... DRAWS <n>	Laufender Punktestand, direkt nach END

Es gibt kein TICK-Broadcast in diesem Protokoll - das Spielfeld existiert nur im Server-Prozess und auf dem Bildschirm. Der Client muss also so gut wie nie lesen, was genau das Problem umgeht, dass FujiNet-Verbindungen bei zu dicht verschachtelten Lese-/Schreibzugriffen unzuverlässig werden können.

3.2 Methode 2: HTTP-Polling (Port 8080)

Verwendet vom C64 (Meatloaf und WiC64) und vom Schneider CPC (M4-Board). Statt einer dauerhaften Verbindung fragt der Client wiederholt zwei einfache HTTP-GET-Routen ab:

Routen

Route	Antwort
GET /join/<plattform>/<name>/<pin-oder-NONE>	"SESSION <id>" + WAIT oder direkt START, falls sofort gepaart
GET /tick/<session>/<richtung-oder-N>	Was auch immer seit dem letzten Poll anliegt (leer, START, END+STATS), oder "ERR UNKNOWN SESSION" wenn die Session vorbei/abgelaufen ist

Eine Session funktioniert wie ein Briefkasten: `/join` legt sie an, jeder `/tick` -Aufruf liefert gleichzeitig eine Richtungsänderung ab und holt ab, was der Server seitdem hinterlegt hat. Nach Spielende bleibt die Session noch **15 Sekunden** gültig (Gnadenfrist), damit auch ein langsamer, realer HTTP-Roundtrip das Endergebnis noch abholen kann, bevor der Server aufräumt.

Wichtige Praxis-Erfahrung: Groß-/Kleinschreibung

Im Praxistest hat sich gezeigt, dass Meatloaf empfangenen HTTP-Inhalt beim Lesen tatsächlich in Kleinbuchstaben umwandelt (keine reine Anzeige-Eigenart, sondern echte Byte-Umwandlung). Der Server vergleicht Session-IDs deshalb bewusst groß-/kleinschreibungs-unabhängig, und die Clients prüfen Schlüsselwörter (START/END/ERR) in beiden Schreibweisen. Bei WiC64 zeigte sich ein verwandtes Problem in umgekehrter Richtung: das Referenzbeispiel für die WiC64-Treiberroutine (`F0T0FIX.C000`) erwartet konsequent **kleingeschriebene** URLs - mit Großschreibung gebaute Join-/Tick-URLs führten zu einem `"/join sieht ok aus, /tick schlägt konsequent mit ERR UNKNOWN SESSION fehl"`-Bug, weil die vermeintliche Session-ID in Wahrheit eine Fehlerantwort war. Bei einer neuen Plattform lohnt es sich, Groß-/Kleinschreibung früh zu testen statt vorauszusetzen.

4. Hardware-Interfaces und Veranstaltung

Jeder Retro-Rechner braucht ein passendes WiFi-/Netzwerk-Interface, um den Server zu erreichen. Projektseiten der eingesetzten Interfaces:

Plattform	Interface	Link
Atari XL/XE	FujiNet	fujinet.online
Commodore 64 (Standard)	Meatloaf	github.com/idolpx/meatloaf
Commodore 64 (experimentell)	WiC64	wic64.de
Schneider/Amstrad CPC	M4-Board	cpcwiki.eu/index.php/M4_Board

Gebaut für [Classic Computing 2026](#), eine Ausstellung des Vereins zum Erhalt klassischer Computer e.V. Termin: 10.–11. Oktober 2026, Halle 10 / CD-Kaserne, Hannoversche Straße 30b, 29221 Celle.

5. Testen ohne echte Hardware

Für den Server gibt es einen minimalen `pygame`-Stub, der es erlaubt, den Server headless laufen zu lassen und per rohem TCP-Socket End-to-End-Tests zu fahren (HELLO senden, WAIT/START abwarten, etc.), ohne ein echtes pygame-Fenster zu benötigen.

Für die BASIC-Clients gibt es je nach Plattform unterschiedlich gute Emulator-Unterstützung - Stand aus der Praxis:

Plattform	Emulator	Stand
Atari (FujiNet)	Fujisan 2.0.5beta	Funktioniert zuverlässig unter Linux. Unter Windows funktioniert dasselbe Setup nicht - Ursache bisher unbekannt.
C64 (WiC64)	VICE	Kann den WiC64-Client emulieren - der experimentelle Client lässt sich damit ohne echte Hardware testen.
CPC (M4)	CPCEmu	Simuliert ein M4-Interface, bisher aber nicht getestet - Status offen.

Für das Verpacken eines BASIC-Programms in ein D64-Disk-Image (z.B. für C64-Tests mit VICE) hat sich [d64-inspector](#) (Autor: P. David Buchan, GPLv3) als nützlich erwiesen. Für die PETSCII-Ansicht nutzt d64-inspector selbst die [C64 TrueType](#)-Fontfamilie (Autor: "Style", style64.org) - beides externe Werkzeuge, nicht Teil dieses Repos. Fertige Referenz-Disk-Images (Atari, CPC, das WiC64-Treiber-Original) liegen unter `disk-images/` im Repo.

Ohne den echten Event-Fotoserver lässt sich `PHOTO_SOURCE = "FTP"` lokal mit einem simplen FTP-Server testen (aus dem Bilderverzeichnis heraus gestartet):

```
# Linux
sudo python -m pyftplib -p 21
# Windows
python -m pyftplib -p 21
```

6. Erweiterung um weitere Retro-Computer

FujiNet existiert nicht nur für den Atari, sondern z.B. auch für den Apple II und für MS-DOS-PCs. Grundsätzlich gilt für eine neue Plattform:

6.1 Die richtige Methode wählen

- **Kann die Netzwerk-Hardware eine dauerhafte TCP-Verbindung aus BASIC heraus offen halten?** Dann Methode 1 (rohes TCP, Port 6502) verwenden - das Protokoll ist bereits generisch gehalten (Plattform-Name wird einfach mitgeschickt) und braucht keine Server-Änderung.
- **Kann sie nur einzelne, einmalige HTTP-Anfragen?** Dann Methode 2 (HTTP-Polling, Port 8080) verwenden - ebenfalls ohne Server-Änderung nutzbar, da die Bridge bereits plattform-neutral aufgebaut ist.

In beiden Fällen ist der Plattform-Name (z.B. "APPLE2" oder "DOS") frei wählbar - er wird nur als Text mitgeschickt und dient der Anzeige.

6.2 Client-seitig nötig

- Joystick/Eingabe auf der Zielplattform lesen (jede Plattform hat hier ihre eigene Methode - beim Atari `STICK()`, beim C64 ein PEEK auf ein CIA-Register, beim CPC `JOY()`). Konvention in diesem Projekt: immer den **ersten** Joystick-Port des Rechners verwenden.
- HELLO/MOVE/BYE senden (Methode 1) oder /join+/tick pollen (Methode 2) - siehe Abschnitt 3 für das exakte Format.
- Optional: PIN-Abfrage vor dem Verbindungsaufbau, für die Foto-Anzeige.

6.3 Server-seitig optional (nur für die Optik)

Damit die neue Plattform im Fenster eine passende Farbe und ein Firmenlogo bekommt, im Server zwei Konfigurationsblöcke ergänzen:

```
PLATFORM_COLORS = {  
    ...  
    "APPLE2": (100, 100, 255), # eigene farbe waehlen  
}  
LOGO_NAME_CANDIDATES = {  
    ...  
    "APPLE2": ["apple", "apple2"],  
}
```

6.4 Ehrliche Erwartung

Jede bisherige Plattform in diesem Projekt hat ihre eigenen, vorher nicht dokumentierten Eigenheiten gezeigt (siehe Abschnitt 3.2 zur Groß-/Kleinschreibung bei Meatloaf und WiC64, oder z.B. dass "PI" ein reserviertes Schlüsselwort in Amstrad-BASIC ist und nicht als eigener Variablenname verwendet werden kann). Für eine neue Plattform lohnt es sich, von Anfang an eine sichtbare Rohantwort-

Diagnose in den Client einzubauen (die rohe Serverantwort einfach mit anzeigen) - das hat sich in der Praxis als der schnellste Weg erwiesen, echte Hardware-Eigenheiten zu finden, statt sie zu erraten.

7. Konfiguration im Überblick

Alle folgenden Einstellungen stehen als klar markierte Konfigurationsblöcke am Anfang von `tron_server.py`:

Block	Zweck
BASE_DIR	Basisordner (Speicherort des Skripts), an dem alle relativen Pfade unten aufgehängt sind
SERVER_BUILD	Versionsnummer - bei jeder Änderung erhöhen, erscheint in Konsole und Fenster
SCROLL_TEXT + SCROLL_*	Demo-Scrolltext (läuft nur im Wartezustand) und sein Aussehen
RESULTS_CSV_PATH	Wo Spielergebnisse geloggt und beim Start wieder eingelesen werden
FTP_HOST / FTP_*	Zugangsdaten zum FTP-Server für Besucherfotos
LOGO_DIR / LOGO_NAME_CANDIDATES	Ordner und Dateinamen für Firmenlogos + Vereinslogo
GRID_W / GRID_H	Größe des Spielfelds
TICK_RATE	Bewegungsgeschwindigkeit (Ticks pro Sekunde) - betrifft alle Plattformen gleichermaßen

Stand: passend zum Server-Build, der zum Zeitpunkt der Erstellung dieses Dokuments aktuell war. Bei größeren Protokolländerungen empfiehlt es sich, dieses Dokument erneut zu erzeugen.